Day 5 — Pandas for Data Engineering

Cleaning, Filtering & Aggregating Real Data — Notes for Recap

---

1. read_csv / read_json

Pandas provides simple functions to load data directly from files into a `DataFrame` — a table-like structure that is the core object you work with in almost every data engineering script.

```
import pandas as pd

df_csv = pd.read_csv("sales.csv")
df_json = pd.read_json("orders.json")

print(df_csv.head())      # preview first 5 rows
print(df_csv.shape)       # (rows, columns)
print(df_csv.columns)     # list of column names
```

Key takeaway: *A DataFrame is essentially an in-memory spreadsheet. Almost every pandas operation you do next — filtering, grouping, merging — happens on top of this structure.*

---

2. Filtering & Sorting

Filtering selects only the rows that match a condition. **Sorting** reorders rows based on one or more column values.

```
# Filtering - only rows where amount > 1000
high_value = df_csv[df_csv["amount"] > 1000]

# Multiple conditions
filtered = df_csv[(df_csv["amount"] > 1000) & (df_csv["region"] == "West")]

# Sorting
sorted_df = df_csv.sort_values("amount", ascending=False)
```

Key takeaway: *Filtering and sorting are the pandas equivalent of SQL's* `WHERE` *and* `ORDER BY` *— the same logic, just written in Python instead of SQL.*

---

3. groupby & Aggregation

`groupby()` splits data into groups based on a column's values, then applies an aggregation (sum, mean, count, etc.) to each group separately — the pandas equivalent of SQL's `GROUP BY`.

```
# Total sales per region
region_totals = df_csv.groupby("region")["amount"].sum()

# Multiple aggregations at once
summary = df_csv.groupby("region")["amount"].agg(["sum", "mean", "count"])

print(region_totals)
print(summary)
```

Key takeaway: *This is one of the most-used pandas operations in real work — answering questions like "total revenue per region" or "average order value per customer" almost always comes down to* `groupby()`*.*

---

4. merge / concat / pivot

merge — combines two DataFrames based on a shared key column (like a SQL JOIN).

```
orders = pd.DataFrame({"order_id": [1, 2], "customer_id": [101, 102]})
customers = pd.DataFrame({"customer_id": [101, 102], "name": ["Akash", "Priya"]})

merged = pd.merge(orders, customers, on="customer_id", how="left")
print(merged)
```

concat — stacks two DataFrames together (rows or columns), useful when combining multiple files of the same shape.

```
jan_sales = pd.read_csv("jan.csv")
feb_sales = pd.read_csv("feb.csv")

combined = pd.concat([jan_sales, feb_sales])   # stacks rows on top of each other
```

pivot — reshapes data from long format into a wide summary table.

```
pivot_table = df_csv.pivot_table(values="amount", index="region", columns="month", aggfunc="sum
```

Key takeaway: `merge` *= SQL JOIN,* `concat` *= stacking files together,* `pivot` *= turning long data into a wide summary — three different ways of reshaping data you'll use constantly.*

---

5. Missing & Duplicate Handling

Real-world data is rarely clean. Pandas gives direct tools to find and handle missing values (`NaN`) and duplicate rows before any analysis happens.

```
# Finding missing values
print(df_csv.isnull().sum())        # count of missing values per column

# Handling missing values
df_csv.dropna()                      # remove rows with any missing value
df_csv.fillna(0)                     # replace missing values with 0
df_csv["amount"].fillna(df_csv["amount"].mean())   # fill with column average

# Handling duplicates
df_csv.duplicated().sum()            # count duplicate rows
df_csv.drop_duplicates()             # remove duplicate rows
```

Key takeaway: *Deciding whether to drop, fill, or flag missing/duplicate data is a judgment call every data engineer has to make — and it should always be a deliberate decision, not something silently ignored.*

---

6. Datetime Ops

Dates often arrive as plain text and need to be converted into pandas' proper datetime type before you can filter by date range, extract the month/year, or calculate differences between dates.

```
df_csv["order_date"] = pd.to_datetime(df_csv["order_date"])

df_csv["year"] = df_csv["order_date"].dt.year
df_csv["month"] = df_csv["order_date"].dt.month
df_csv["day_of_week"] = df_csv["order_date"].dt.day_name()

# Filter to a specific date range
recent = df_csv[df_csv["order_date"] >= "2026-01-01"]
```

Key takeaway: *Always convert date columns with* `pd.to_datetime()` *before working with them — comparing or filtering on plain text dates gives wrong or inconsistent results.*

Day 5 Summary — Pandas for Data Engineering

Concept	One-Line Recap
read_csv / read_json	Loading data from files into a DataFrame
Filtering & Sorting	Selecting rows by condition and reordering them - like WHERE and ORDER BY
groupby & Aggregation	Splitting data into groups and summarizing each - like SQL GROUP BY
merge / concat / pivot	Joining, stacking, and reshaping DataFrames
Missing & Duplicate Handling	Finding and deliberately handling NaN values and duplicate rows
Datetime Ops	Converting text to real dates so you can filter, extract, and compare properly

Overall takeaway: *Pandas is the tool you'll reach for constantly in data engineering - almost every cleaning and aggregation task before data reaches a warehouse or dashboard runs through these six operations.*